

HDOC ASSIGNMENT DAY 23

Menu-Driven Pattern Printing Using Switch Statement, Nested Loops and Functions in C

1. Problem Understanding

The task is to develop a menu-driven C program that displays four basic square/rectangular patterns selected by the user. A switch statement is used to perform the selected operation. Nested loops are used without arrays, and each pattern is implemented in a separate parameterless function with no return value.

- Option 1: Input n and print a solid square pattern containing n rows and n columns.
- Option 2: Input m rows and n columns and print a solid rectangular pattern.
- Option 3: Input m rows and n columns and print a row-number pattern in which every element of a row equals its row number.
- Option 4: Input m rows and n columns and print a column-number pattern in which every element of a column equals its column number.
- Use nested loops for all patterns and validate the dimensions so that only positive values are accepted.

2. Standard Test Cases

Test Case	Complete Inputs	Expected Result	Status
TC1	Choice = 1 n = 4	A 4 x 4 solid square of * symbols is printed.	Pass
TC2	Choice = 2 m = 3, n = 5	A 3 x 5 solid rectangular pattern of * symbols is printed.	Pass
TC3	Choice = 3 m = 4, n = 4	Four rows are printed; each row contains its row number four times.	Pass
TC4	Choice = 4 m = 4, n = 4	Each row prints the column numbers 1 2 3 4.	Pass
TC5	Choice = 1 n = 0 (invalid), then 3	Invalid size is rejected; after entering 3, a 3 x 3 square is printed.	Pass
TC6	Choice = 7	An invalid-choice error message is displayed.	Pass

3. Algorithm

1. Start.
2. Declare integer variable choice.
3. Display the menu with four pattern choices.
4. Input choice.
5. Use a switch statement to process the selected choice.
 - 5.1 For choice 1, call solidSquare(). Input n, validate $n > 0$, and use nested loops to print n rows and n columns of * symbols.
 - 5.2 For choice 2, call solidRectangle(). Input m and n, validate both as positive, and use nested loops to print m rows and n columns of * symbols.

5.3 For choice 3, call `rowNumberPattern()`. Input `m` and `n`, validate both as positive, and use nested loops. For each row `i`, print `i` exactly `n` times.

5.4 For choice 4, call `columnNumberPattern()`. Input `m` and `n`, validate both as positive, and use nested loops. In every row, print the column number `j` from 1 to `n`.

5.5 For any other choice, display an appropriate invalid-choice error message.

6. Stop.

4. C Program

```
#include <stdio.h>

void solidSquare();
void solidRectangle();
void rowNumberPattern();
void columnNumberPattern();
int main()
{
    int choice;
    printf("1. Solid square pattern\n");
    printf("2. Solid rectangular pattern\n");
    printf("3. Row-number pattern\n");
    printf("4. Column-number pattern\n");
    printf("Enter your choice: ");
    scanf("%d", &choice);
    switch(choice)
    {
        case 1: solidSquare();
        break;
        case 2: solidRectangle();
        break;
        case 3: rowNumberPattern();
        break;
        case 4: columnNumberPattern();
        break;
        default:
            printf("Invalid menu choice. Please enter 1 to 4.\n");
    }
    return 0;
}
```

```
void solidSquare()
{
    int n, i, j;
    do
    {
        printf("Enter number of rows: ");
        scanf("%d", &n);
        if(n <= 0)
            printf("Invalid input. Enter a positive value.\n");
    }
    while(n <= 0);

    for(i = 1; i <= n; i++)
    {
        for(j = 1; j <= n; j++)
            printf("* ");
        printf("\n");
    }
}

void solidRectangle()
{
    int m, n, i, j;
    do
    {
        printf("Enter rows and columns: ");
        scanf("%d %d", &m, &n);
        if(m <= 0 || n <= 0)
            printf("Invalid input. Enter positive values.\n");
    }
    while(m <= 0 || n <= 0);
    for(i = 1; i <= m; i++)
    {
        for(j = 1; j <= n; j++)
            printf("* ");
        printf("\n");
    }
}
```

```

    }
}
void rowNumberPattern()
{
    int m, n, i, j;
    do
    {
        printf("Enter rows and columns: ");
        scanf("%d %d", &m, &n);
        if(m <= 0 || n <= 0)
            printf("Invalid input. Enter positive values.\n");
    }
    while(m <= 0 || n <= 0);
    for(i = 1; i <= m; i++)
    {
        for(j = 1; j <= n; j++)
            printf("%d ", i);
        printf("\n");
    }
}
void columnNumberPattern()
{
    int m, n, i, j;
    do
    {
        printf("Enter rows and columns: ");
        scanf("%d %d", &m, &n);
        if(m <= 0 || n <= 0)
            printf("Invalid input. Enter positive values.\n");
    }
    while(m <= 0 || n <= 0);
    for(i = 1; i <= m; i++)
    {
        for(j = 1; j <= n; j++)
            printf("%d ", j);
    }
}

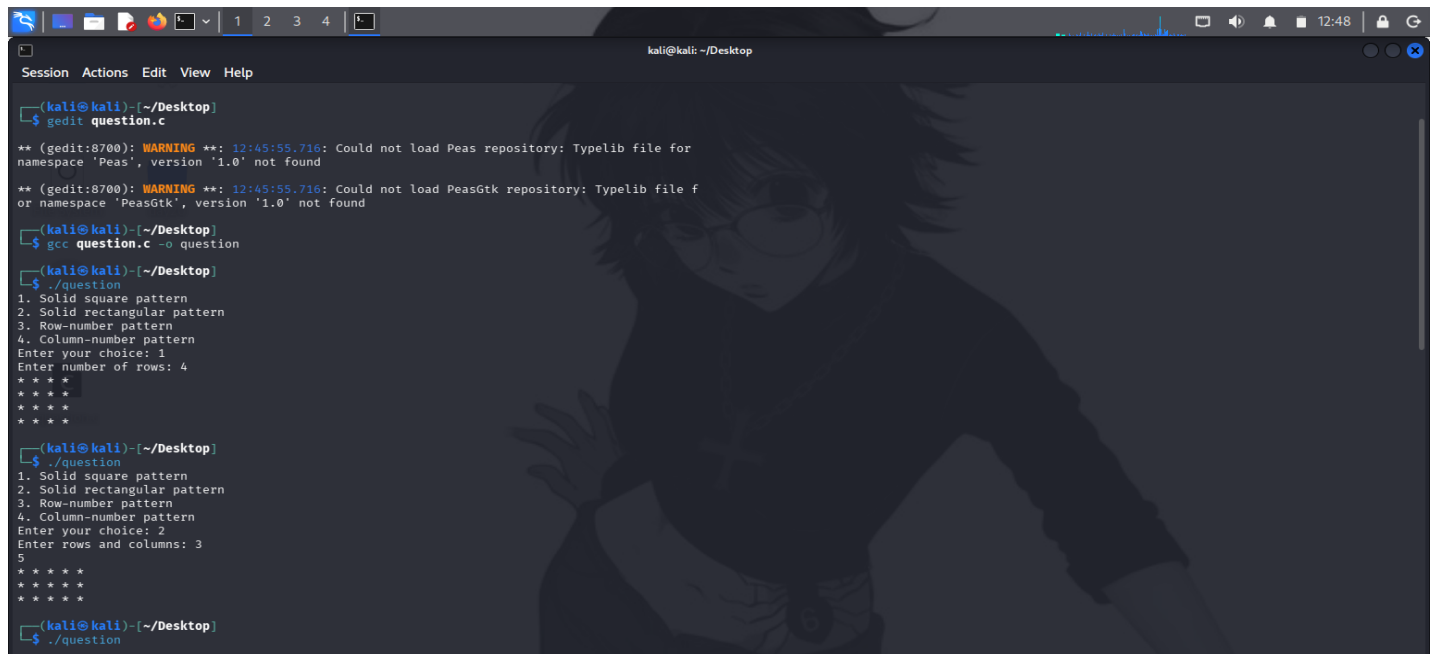
```

```

printf("\n");
}
}

```

5. Compilation and Execution Evidence



```

kali@kali: ~/Desktop
Session Actions Edit View Help

(kali@kali)~/Desktop
$ gedit question.c

** (gedit:8700): WARNING **: 12:45:55.716: Could not load Peas repository: Typelib file for namespace 'Peas', version '1.0' not found

** (gedit:8700): WARNING **: 12:45:55.716: Could not load PeasGtk repository: Typelib file f
or namespace 'PeasGtk', version '1.0' not found

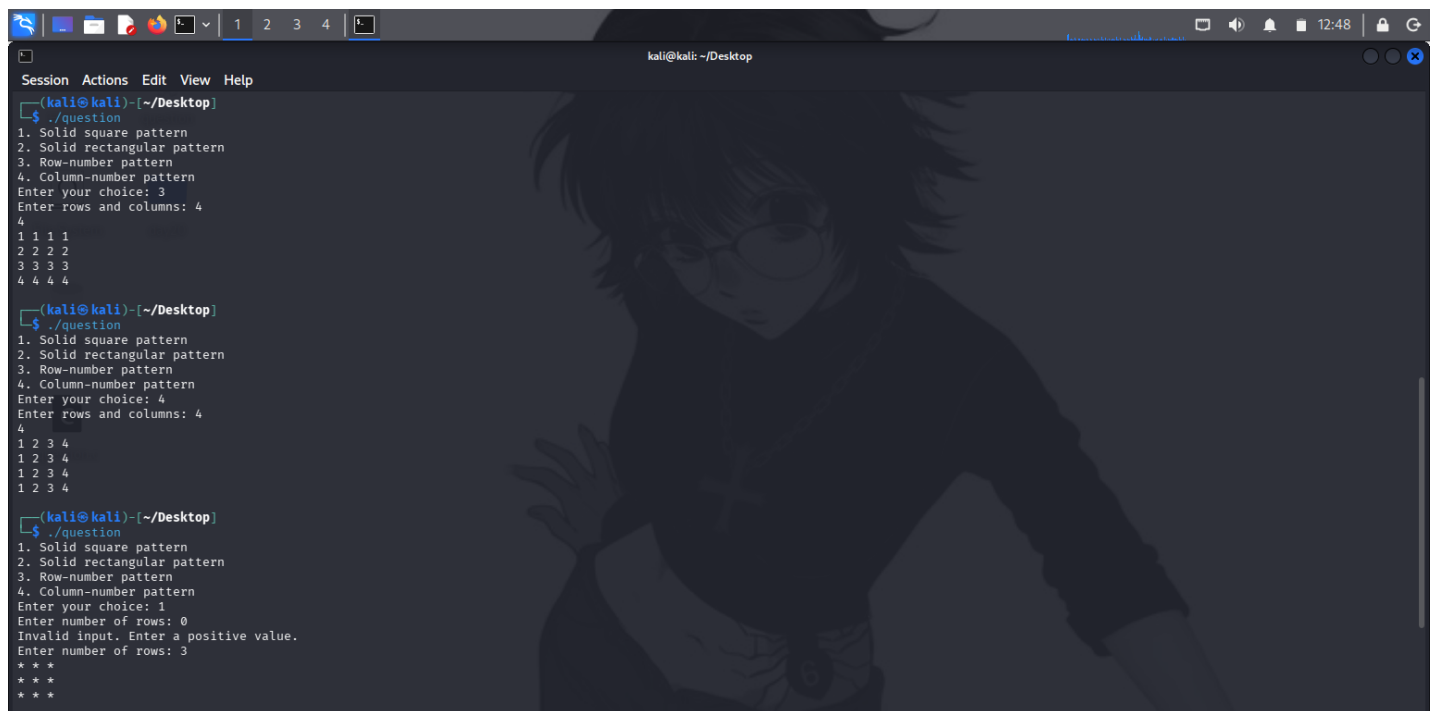
(kali@kali)~/Desktop
$ gcc question.c -o question

(kali@kali)~/Desktop
$ ./question
1. Solid square pattern
2. Solid rectangular pattern
3. Row-number pattern
4. Column-number pattern
Enter your choice: 1
Enter number of rows: 4
* * * *
* * * *
* * * *
* * * *

(kali@kali)~/Desktop
$ ./question
1. Solid square pattern
2. Solid rectangular pattern
3. Row-number pattern
4. Column-number pattern
Enter your choice: 2
Enter rows and columns: 3
5
* * * *
* * * *
* * * *

(kali@kali)~/Desktop
$ ./question

```



```

kali@kali: ~/Desktop
Session Actions Edit View Help

(kali@kali)~/Desktop
$ ./question
1. Solid square pattern
2. Solid rectangular pattern
3. Row-number pattern
4. Column-number pattern
Enter your choice: 3
Enter rows and columns: 4
4
1 1 1 1
2 2 2 2
3 3 3 3
4 4 4 4

(kali@kali)~/Desktop
$ ./question
1. Solid square pattern
2. Solid rectangular pattern
3. Row-number pattern
4. Column-number pattern
Enter your choice: 4
Enter rows and columns: 4
4
1 2 3 4
1 2 3 4
1 2 3 4
1 2 3 4

(kali@kali)~/Desktop
$ ./question
1. Solid square pattern
2. Solid rectangular pattern
3. Row-number pattern
4. Column-number pattern
Enter your choice: 1
Enter number of rows: 0
Invalid input. Enter a positive value.
Enter number of rows: 3
* * *
* * *
* * *

```

```
kali@kali: ~/Desktop
Session Actions Edit View Help

(kali@kali)~/Desktop
$ ./question
1. Solid square pattern
2. Solid rectangular pattern
3. Row-number pattern
4. Column-number pattern
Enter your choice: 4
Enter rows and columns: 4
4
1 2 3 4
1 2 3 4
1 2 3 4
1 2 3 4

(kali@kali)~/Desktop
$ ./question
1. Solid square pattern
2. Solid rectangular pattern
3. Row-number pattern
4. Column-number pattern
Enter your choice: 1
Enter number of rows: 0
Invalid input. Enter a positive value.
Enter number of rows: 3
* * *
* * *
* * *

(kali@kali)~/Desktop
$ ./question
1. Solid square pattern
2. Solid rectangular pattern
3. Row-number pattern
4. Column-number pattern
Enter your choice: 7
Invalid menu choice. Please enter 1 to 4.

(kali@kali)~/Desktop
$
```

6. Verification of Test Cases

Test Case	Complete Inputs	Expected Result	Actual Result	Status
TC1	Choice = 1 n = 4	4 x 4 solid square	4 x 4 solid square displayed correctly	Pass
TC2	Choice = 2 m = 3, n = 5	3 x 5 solid rectangle	3 x 5 solid rectangle displayed correctly	Pass
TC3	Choice = 3 m = 4, n = 4	1 1 1 1 2 2 2 2 3 3 3 3 4 4 4 4	Output matches the expected row-number pattern	Pass
TC4	Choice = 4 m = 4, n = 4	1 2 3 4 repeated for 4 rows	Output matches the expected column-number pattern	Pass
TC5	Choice = 1 n = 0, then 3	Reject invalid n; print 3 x 3 square	Input validation works as expected	Pass
TC6	Choice = 7	Invalid-choice message	Invalid menu choice is handled correctly	Pass